

Hotel Management System

Mandatory Assignment 1 & 2

Relational Database Design & Implementation

<https://github.com/Codemagic420/HotelManagementBackend>
<https://github.com/Driconaari/HotelSQLGroupWork>

Group Members:

Asger Valdemar Bergøe (asbe0002@stud.ek.dk)

Magnus Sørensen (magn554n@stud.ek.dk)

Sophus Ingi Sophusson (soso0001@stud.ek.dk)

Joel Darko-Martinez (joda0002@stud.ek.dk)

KEA - Copenhagen School of Design and Technology

Date: April 2026

Hotel Management System – Mandatory Assignment 1

1. Introduction

The goal of this project is to design and implement a relational database for a hotel management system. The system manages guests, room bookings, seasonal pricing (rates), room maintenance (cleaning status), and automated billing for minibar and services.

The implementation consists of 14 normalized tables that prevent data redundancy and ensure referential integrity across the reservation and housekeeping modules.

1.1 Development Workflow & Tools

IDE: DataGrip for writing and testing SQL with real-time schema inspection optimized for MySQL 8.0.

Version Control: GitHub repository with Feature Branch workflow and Pull Requests for code review.

Scripts: Idempotent SQL using IF NOT EXISTS, allowing re-runnable execution without conflicts.

2. Relational Database

2.1 Introduction to Relational Databases

A relational database organizes data into tables with rows and columns. Each table represents an entity, and relationships between tables are established through primary and foreign keys. This project uses MySQL 8.0, supporting ACID transactions, stored procedures, triggers, and views.

2.2 Database Design

2.2.1 Entity/Relationship Model

The database was designed using a three-stage process: 1) Conceptual model — brainstorming entities and relationships, 2) Logical model — defining attributes and cardinalities, 3) Physical model — implementing in MySQL with specific data types.

Core Entities: GUEST, RESERVATION, ROOM, ROOM_TYPE, BILL, BILL_ITEM, SEASON_RATE, INVENTORY_ITEM, EXTRA_SERVICE, CLEANER, ROOM_CLEANING_TASK

Junction Tables: RESERVATION_GUEST, ROOM_CLEANING_ASSIGNMENT. See Entity Relationship Diagram (2).pdf in the repository.

2.2.2 Normalization Process

The database is normalized to Third Normal Form (3NF): **1NF**: Atomic values, unique rows. **2NF**: Non-key attributes depend on full key. **3NF**: No transitive dependency.

2.3 Physical Data Model

2.3.1 Data Types

Column Type	MySQL Type	Usage
Primary Keys	INT AUTO_INCREMENT	All ID columns
Names/Text	VARCHAR(50-255)	first_name, last_name, etc.
Prices	DECIMAL(10,2)	Room rates, billing
Dates	DATE	check_in, check_out
Status / Booleans	ENUM / BOOLEAN	room_status, is_paid

2.3.2 Primary/Foreign Keys & 2.3.3 Indexes

All tables use single-column AUTO_INCREMENT PK. FKs enforce referential integrity with ON DELETE CASCADE where appropriate. **Indexes**: idx_guest_email (fast email lookup), idx_ref_no (reservation lookup), idx_res_status (filter by status).

2.3.4 Constraints & Referential Integrity

NOT NULL for required data, UNIQUE constraints on email/reference_no/room_number, CHECK constraint ensuring check_out_date > check_in_date, FOREIGN KEY with cascade rules.

2.4 Stored Objects

fn_GetRoomRate: Automates lookup of room rate prices per season. **sp_CalculateFinalBill**: Calculates room nights × rate, adds minibar/service items, updates BILL table. **tr_AfterCheckout**: Automatically marks room status → Vacant, clean status → Dirty, creates cleaning task. **vw_HousekeepingList**: Shows only data needed by cleaners.

Conclusion (Assignment 1)

This assignment delivered a normalized 14-table MySQL database handling core hotel operations including guests, bookings, seasonal pricing, housekeeping, and automated billing.

Hotel Management System – Mandatory Assignment 2

3. Backend Application

3.1 Introduction

Mandatory Assignment 2 extends Assignment 1 by implementing a full backend application using Java Spring Boot. The backend connects to the existing MySQL database and exposes RESTful endpoints with authentication and authorization. "Installation instructions are available in the README.md of the backend repository."

3.2 Development Tools & Workflow

Spring Boot 4.0.5 (framework), Spring Data JPA (ORM), Spring Security 7 (auth), Swagger UI (API docs), IntelliJ IDEA (IDE), GitHub (version control with same branching strategy as Assignment 1).

3.3 Backend Architecture

Layered Architecture: Client (Swagger/Postman) → Controller Layer → Service Layer → Repository Layer (JPA) → MySQL Database

Layers: Controllers (REST endpoints), Services (business logic), Repositories (ORM data access), Models (entities mapping to SQL), Security (login + roles), Migration (SQL → MongoDB/Neo4j).

3.4 Object-Relational Mapping (ORM)

Hibernate maps Java entities directly to SQL tables. **Mapped Entities:** Guest, Room, Reservation, Bill, BillItem, UserAccount (security). This satisfies: "Use an ORM tool such as Hibernate."

3.5 CRUD Implementation

Method	Endpoint	Description
GET	/api/guests	Fetch all guests
POST	/api/guests	Create guest
DELETE	/api/guests/{id}	Remove guest

Full CRUD also implemented for Room, Reservation, and Bill entities.

3.6 Authentication & Security

Spring Security 7: UserAccount entity, Roles (ADMIN, STAFF, CLEANER), Protected routes (e.g., /api/guests/**), Form login, BCrypt password encoding. **SQL Injection Prevention:** JPA uses prepared statements, no string-built SQL. **Backup:** mysqldump -u root -p hotel_db > backup.sql

3.7 Swagger UI

Acts as the "frontend" for this assignment. URL: http://localhost:8080/swagger-ui/index.html. Provides interactive testing, auto-generated documentation, and schema visualization. Satisfies: "Use Postman or Swagger UI to simulate the frontend."

4. Document Database (MongoDB)

4.1 Introduction

MongoDB is a NoSQL document database storing data in flexible, JSON-like documents (BSON). **Key Design Principle:** Denormalization for performance — related entities are embedded within parent documents rather than referenced separately.

4.2 Database Design

Collections

Collection	Description & Use Cases
rooms	Room inventory & status. Fields: roomId, roomNumber, type, occupied
reservations	Bookings with embedded guest & room documents. Single read instead of 3 joins.
bills	Invoices with embedded items array. Atomic bill + items retrieval.

Document Structure — reservations (Real Data)

```
{ "_id": ObjectId("69ecae47394acffld93efbcf"), "reservationId": 1,
  "status": "CONFIRMED", "checkInDate": "2024-06-01", "checkOutDate": "2024-06-05",
  "guest": { "guestId": 1, "firstName": "John", "lastName": "Doe",
    "email": "john.doe@example.com" },
  "room": { "roomId": 3, "roomNumber": "201", "type": "Double" } }
```

Migrated guests: John Doe (Room 201), Jane Smith (Room 101), Bob Johnson (Room 301)

4.3 Why MongoDB? (Design Rationale)

Performance: Single document reads eliminate join operations. **Flexibility:** Schema-less allows easy field additions. **Real-time:** Fast reads perfect for dashboards. **Caching:** Entire business objects cached together.

5. Graph Database (Neo4j)

5.1 Introduction

Neo4j stores data as nodes and relationships. **Key Design Principle:** Explicit relationship modeling — traverses relationships directly instead of using JOINS.

5.2 Database Design

Nodes (11 total)

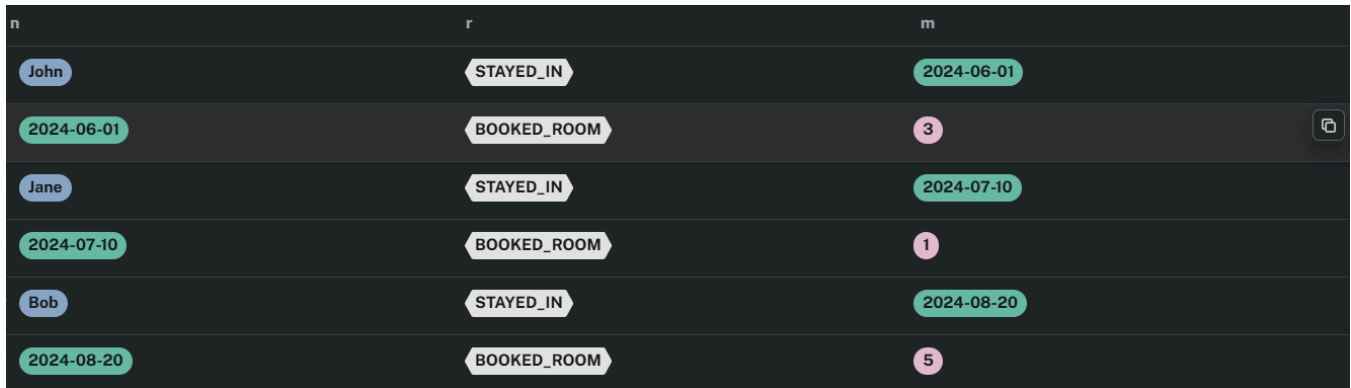
Label	Properties	Count
:Guest	guestId, firstName, lastName, email	3 (John, Jane, Bob)
:Room	roomId, roomNumber, type	5
:Reservation	reservationId, status, checkIn, checkOut	3

Relationships (6 total)

Relationship	Pattern	Count
STAYED_IN	(:Guest)-[:STAYED_IN]->(:Reservation)	3
BOOKED_ROOM	(:Reservation)-[:BOOKED_ROOM]->(:Room)	3

Graph Visualization (Real Data)

Jane —STAYED_IN→ Res (2024-07-10) —BOOKED_ROOM→ Room 1 (Single)
Bob —STAYED_IN→ Res (2024-08-20) —BOOKED_ROOM→ Room 5 (Suite)
John —STAYED_IN→ Res (2024-06-05) —BOOKED_ROOM→ Room 3 (Double)



visualisation.png

5.3 Why Neo4j? (Design Rationale)

Relationships: First-class relationship support. **Traversal:** Fast path queries (guest → reservations → rooms). **Analysis:** Find guests who stayed multiple times, room popularity. **Pattern Matching:** Powerful Cypher query language.

6. Migration Application

DataMigrator.java handles one-time migration from MySQL to MongoDB and Neo4j. **Endpoint:** POST /api/migrate (ADMIN role required)

6.1 Data Mapping

MySQL (Source)	MongoDB	Neo4j
GUEST table	Embedded in reservations	:Guest node
ROOM table	rooms collection	:Room node
RESERVATION table	reservations collection	:Reservation node
Foreign keys (JOINS)	Embedded documents	Relationships

7. Conclusion

Assignment 1: Delivered a normalized 14-table MySQL database handling guests, bookings, seasonal pricing, housekeeping, and automated billing.

Assignment 2: Expanded to a functional backend with CRUD, security, ORM, Swagger UI, and working migration to MongoDB (3 collections) and Neo4j (11 nodes, 6 relationships). Data successfully migrated: John Doe, Jane Smith, Bob Johnson with their reservations.